

On Characterizing the Laplace–Beltrami Operator, Learning Stiffness Weights, and Variational Training

A conversation with Claude

May 7, 2026

Contents

1	Does agreement on coordinate functions imply my operator is the LBO?	1
2	Action on a dense subspace	2
3	Training a network to predict per-vertex stiffness weights	2
3.1	Setup	3
3.2	Loss design	3
3.3	Test function families	3
3.4	Generating Monge patches	3
3.5	Loss enhancements	4
3.6	Curriculum	4
4	What are monomials?	4
5	Inference: assembling the Laplacian from per-vertex predictions	4
6	Can we deduce the gradient operator from W?	5
7	Predicting the gradient first, then assembling the Laplacian	5
7.1	Architecture	5
7.2	Training loss	5
7.3	Assembly	6
7.4	Structural advantages	6
8	Sparsity considerations	6
9	Unpacking the $G^\top M_F G$ construction	6
9.1	The 1D analog	6
9.2	The general case	7
9.3	The action formula	7
10	Patch-local training of an assembled-W loss	7
11	A pure-norm shortcut: $w_{ij} = \ G_{:,j}^{(i)}\ ^2$?	8

12	What the stiffness weights actually are when you assemble $G^\top M_F G$	8
13	Continuous-space training: opportunities from analytic data	8
14	Avenue 2: spectral / functional supervision	9
15	Avenue 4: the network as a learned discretization scheme	10
16	Avenue 3: variational training via the Dirichlet energy	10
17	Computing the continuous Dirichlet energy on a Monge patch	11
17.1	Setup	11
17.2	Pull back to parameter domain	11
17.3	Restricted function and gradient	11
17.4	Final integral	11
17.5	Numerical evaluation	11
17.6	Worked example	12
17.7	Implementation	12
18	Forward-pass design: predict G or W ?	12
18.1	Two main options	12
18.2	Recommendation: Option B	12
18.3	Mass matrix	12
18.4	Architecture sketch	12
19	Computing $\widehat{E}^{(\ell)}$: a clarification	13
20	Does energy minimization guarantee tangent gradients?	13
21	Bilinear-form vs. energy supervision	14
22	Choice of Monge surface family	14
23	Implementing GP-like sampling	15

1 Does agreement on coordinate functions imply my operator is the LBO?

Consider I have an operator that works on a 3D shape / 2-manifold or surface in 3D. If the action of that operator on the coordinate functions of that shape agrees with the action of the Laplace–Beltrami operator on that shape, does it mean that my operator is the Laplace–Beltrami operator?

No, not in general. Agreement on the three coordinate functions x, y, z is a necessary but far from sufficient condition. The Laplace–Beltrami operator Δ_g is a linear operator on an infinite-dimensional space; matching it on three specific functions only pins down the operator on a 3-dimensional subspace of its domain.

A concrete way to see the under-determination. Suppose L is your operator and $L(x^i) = \Delta_g(x^i)$ for $i = 1, 2, 3$. Define $D = L - \Delta_g$. Then D is any linear operator that vanishes on the

coordinate functions. The space of such operators is huge — e.g., any second-order differential operator engineered to annihilate x, y, z but act nontrivially on x^2 or $\sin(x)$. There are infinitely many such D .

The “mean curvature normal” identity is the trap. The well-known identity

$$\Delta_g \mathbf{X} = -2H\mathbf{n},$$

is genuinely beautiful, but it’s only a statement about how Δ_g acts on *one specific triple of functions*. Reproducing $-2H\mathbf{n}$ doesn’t characterize the operator.

What would actually suffice?

1. *Strong structural assumptions on L* (second-order, no zeroth-order term, same principal symbol, derivation-of-derivations type) plus agreement on coordinates and constants.
2. *Spectral characterization*: full eigenvalue/eigenfunction system or heat kernel for all $t > 0$.
3. *Variational characterization*: L arises as the gradient of the Dirichlet energy w.r.t. the L^2 inner product induced by g .
4. *Action on a dense subspace* of the function space.

So matching $L\mathbf{X} \approx -2H\mathbf{n}$ is necessary but not sufficient — the gap is genuinely large.

2 Action on a dense subspace

A bounded linear operator on a Banach space is determined by its values on any dense subspace by continuity. So if $L = \Delta_g$ on a dense subspace of the appropriate function space (and both are continuous in the corresponding topology), they agree everywhere.

The catch: Δ_g is unbounded on $L^2(M)$. Its domain is $H^2(M) \subsetneq L^2(M)$. So:

Option A: Stronger topology. $\Delta_g : H^2(M) \rightarrow L^2(M)$ is bounded. Density in H^2 on a subspace where $L = \Delta_g$ extends agreement to all of H^2 .

Option B: Closed graph / closability. If L and Δ_g are closed and agree on a *core* of Δ_g , agreement extends to the full domain. Standard tool in spectral theory of unbounded operators.

Concrete dense subspaces. **Smooth functions** $C^\infty(M)$ are dense in every H^k and form a core for Δ_g .

Polynomials in ambient coordinates. For $M \subset \mathbb{R}^3$ compact, polynomials in x, y, z restricted to M form a subalgebra of $C^\infty(M)$. By Stone–Weierstrass, dense in $C(M)$, hence dense in $L^2(M)$. Concretely checkable.

Eigenfunctions of Δ_g . Form an orthonormal basis but circular if you’re trying to characterize Δ_g .

Why polynomials are the “right” answer. For point clouds in $\mathbb{R}^3$:

1. Tied to the embedding (what your network sees).
2. Generalizes the coordinate-function test — coordinate functions are degree 1; you need all degrees for density.
3. Connects to algebraic structure via the Leibniz identity

$$\Delta_g(fg) = f \Delta_g g + g \Delta_g f + 2\langle \nabla_g f, \nabla_g g \rangle.$$

A practical takeaway: evaluating $L(p)$ vs. $\Delta_g(p)$ for low-degree polynomial test functions on canonical shapes (sphere with $\Delta_{S^2} Y_\ell^m = -\ell(\ell+1)Y_\ell^m$, etc.) gives a graded family of stringent tests.

3 Training a network to predict per-vertex stiffness weights

Train a network that, given a local 3D patch, returns stiffness weights, so we can predict the local LBO action and assemble a Laplacian at inference. We can generate analytic Monge patches with closed-form Δ_g .

3.1 Setup

Network f_θ : patch (center v_0 , k -NN neighborhood) $\rightarrow$ weights $\{w_{0j}\}_{j=1}^k$ such that

$$(Lu)(v_0) \approx \sum_{j=1}^k w_{0j} (u(v_j) - u(v_0)).$$

The “minus $u(v_0)$ ” form annihilates constants automatically — bake it in architecturally.

3.2 Loss design

Single-function loss is under-constrained: with k neighbors, weights have k degrees of freedom but each test function gives only one constraint. Sample m test functions per patch:

$$\mathcal{L}_{\text{patch}} = \frac{1}{m} \sum_{\ell=1}^m \left(\sum_j w_{0j} (u^{(\ell)}(v_j) - u^{(\ell)}(v_0)) - \Delta_g u^{(\ell)}(v_0) \right)^2.$$

With $m \gg k$ for an overdetermined system.

3.3 Test function families

Polynomials in ambient coordinates (degrees up to d) — the natural choice via the density argument.

Heat kernel / Gaussian bumps — nice locality, multi-scale.

Spherical harmonics on canonical patches — orthogonal but more setup.

I’d lean toward polynomials: easy to differentiate, span densely, give graded curriculum.

3.4 Generating Monge patches

Patches as graphs of analytic functions:

$$\Sigma = \{(u, v, h(u, v)) : (u, v) \in U \subset \mathbb{R}^2\}.$$

The metric and Δ_g have closed-form expressions in h_u, h_v :

$$g = \begin{pmatrix} 1 + h_u^2 & h_u h_v \\ h_u h_v & 1 + h_v^2 \end{pmatrix}, \quad \det g = 1 + h_u^2 + h_v^2.$$

Sample h from a parametric family (random polynomials, GP samples, named surfaces) for exact ground truth via autodiff.

Pitfall: extrinsic vs. intrinsic Laplacian. For ambient u on Σ :

$$\Delta_{\mathbb{R}^3} u|_{\Sigma} = \Delta_g(u|_{\Sigma}) + H \partial_{\mathbf{n}} u + \partial_{\mathbf{n}}^2 u.$$

You want $\Delta_g(u|_{\Sigma})$, the intrinsic version. The Monge formulation handles this naturally.

3.5 Loss enhancements

Symmetry / consistency. The discrete LBO should have W symmetric. Penalize $|w_{ij}^{\text{from } i} - w_{ji}^{\text{from } j}|$.

Locality / decay prior. Soft regularization toward decay with distance.

Spectral loss. Periodically assemble L on canonical small meshes and penalize eigenvalue/eigenfunction error against analytic spectra.

Sign and definiteness. Encourage PSD-ness.

3.6 Curriculum

Start with low-degree test functions on near-flat patches; progressively introduce higher degrees and curvatures. Vary k , neighborhood radius, anisotropic distributions, and noise — main motivation for a learned operator is robustness.

4 What are monomials?

A **monomial** is a product of variables raised to non-negative integer powers, possibly with a coefficient. In (x, y, z) :

$$x^a y^b z^c, \quad \text{degree} = a + b + c.$$

Counts: degree 0: 1 (one); degree 1: x, y, z (three); degree 2: six; degree 3: ten; degree 4: fifteen. Total up to degree d in n variables: $\binom{n+d}{d}$. For $n = 3, d = 4$: $\binom{7}{4} = 35$.

Every polynomial of degree $\leq d$ is a unique linear combination of these monomials, so they form a natural basis. Agreement on the basis implies agreement on the span.

5 Inference: assembling the Laplacian from per-vertex predictions

For each vertex v_i on a 3D shape, run the network on its local patch to get weights $\{w_{i,j}\}_{j=1}^k$, scatter into a sparse $W \in \mathbb{R}^{n \times n}$:

$$W_{i,i_j} = w_{i,i_j}, \quad W_{ii} = -\sum_j w_{i,i_j}.$$

Practical subtleties. Symmetry. Network sees patches independently; predicted $w_{i \rightarrow j}$ may not equal $w_{j \rightarrow i}$. Symmetrize: $W_{\text{sym}} = \frac{1}{2}(W + W^\top)$, then recompute the diagonal.

Mass matrix. The full LBO is $L = M^{-1}W$. Compute M from local areas (Voronoi, density estimation). For eigenvalue problems, solve the generalized $W\phi = \lambda M\phi$.

Sign convention. Pick one and be consistent.

Cost. Dominated by network forward passes, fully batchable.

Boundary handling. Boundary vertices have truncated neighborhoods; train explicitly with boundary patches or apply different rules there.

6 Can we deduce the gradient operator from W ?

Partially, but not uniquely, and not from W alone. Continuously, $\Delta_g f = -\text{div}_g \nabla_g f$ via the Dirichlet energy

$$E(f) = \frac{1}{2} \int_M \|\nabla_g f\|^2 dV_g.$$

Discretely, the FEM construction goes the other way: $W = G^\top M_F G$ from gradient G and tangent-space mass M_F .

The ambiguity. If $W = G^\top M_F G$ is one factorization, $\tilde{G} = QG$ is another for any orthogonal Q (with $Q^\top M_F Q = M_F$). So W determines the bilinear form (Dirichlet inner product), but not gradient *vectors*.

What you can recover.

- The squared-norm-of-gradient quadratic form: $f^\top W f \approx \int \|\nabla f\|^2$.
- Pairwise gradient inner products: $f^\top W h \approx \int \langle \nabla f, \nabla h \rangle$.
- Whether a function is harmonic.

What you can't. Absolute gradient direction at any vertex; tangent frames; embedding-dependent information.

To get a usable gradient operator, you need the geometry $\{v_i\}$ in addition to W — e.g., PCA + least-squares fit per vertex.

7 Predicting the gradient first, then assembling the Laplacian

If you predict G instead of W , you get structural advantages.

7.1 Architecture

Per patch, network outputs $G^{(i)} \in \mathbb{R}^{3 \times k}$ (or $2 \times k$ in tangent frame):

$$\nabla_g f(v_i) \approx \sum_{j=1}^k G_{:,j}^{(i)} (f(v_{i_j}) - f(v_i)).$$

The $f(v_j) - f(v_i)$ form annihilates constants. Predicting in ambient $\mathbb{R}^3$ with a soft tangency penalty (or hard projection at inference) is cleaner than committing to a tangent frame.

7.2 Training loss

For each test function $u^{(\ell)}$:

$$\hat{g}^{(\ell)} = \sum_j G_{:,j}^{(i)} (u^{(\ell)}(v_j) - u^{(\ell)}(v_i)).$$

Loss compares to analytic $\nabla_g u^{(\ell)}(v_0)$, plus a tangency penalty.

7.3 Assembly

Build sparse $G \in \mathbb{R}^{3n \times n}$, diagonal mass M_F (block-diagonal $A_i I_3$), and $W = G^\top M_F G$.

7.4 Structural advantages

- **Symmetry by construction.**
- **PSD by construction:** $f^\top W f = (Gf)^\top M_F (Gf) \geq 0$.
- **Constants in kernel:** $W \mathbf{1} = G^\top M_F G \mathbf{1} = 0$.
- **Gradient operator as byproduct.**
- **Variational interpretation preserved.**

Sparsity cost. The pattern of W is 2-hop: $W_{ij} \neq 0$ iff v_i, v_j share a common stencil. With $k = 20$ neighbors: ~ 100 – 400 nonzeros per row instead of ~ 20 . A 5 – $20\times$ density blowup.

8 Sparsity considerations

The 2-hop density of $W = G^\top M_F G$ is real but tractable:

Path 1: Minimal gradient stencils. Predict gradients from a $k_G = 3$ stencil (matching triangle case). Less robust but recovers cotangent-like sparsity.

Path 2: Virtual triangulation. Local 2D Delaunay in tangent plane; per-virtual-triangle gradients. Standard FEM sparsity, but commits to a connectivity.

Path 3: Post-sparsification. Compute full W , project onto 1-hop pattern with sparsity-preserving constraints. Memory peaks at dense level.

Path 4: Decoupled-neighbor approximation. Drop cross-patch contributions, keep only diagonal terms. Gives $w_{ij} \propto -A_i \|G_{:,j}^{(i)}\|^2$ (symmetrized), 1-hop sparse.

Path 5: Predict W directly. No structural guarantees beyond what's learned, but native sparsity.

9 Unpacking the $G^\top M_F G$ construction

9.1 The 1D analog

For 4 points on a line, edge gradient G (slope per edge) gives:

$$G = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 0 & -1 & 1 \end{pmatrix}, \quad G^\top G = \begin{pmatrix} 1 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 1 \end{pmatrix}.$$

This is the standard discrete second-difference Laplacian. $G^\top G$ does the work — no magic.

Why $G^\top G$? The quadratic form is

$$f^\top W f = \|Gf\|^2 = \sum_e (f_{i+1} - f_i)^2,$$

the discrete Dirichlet energy. W is the Hessian of this convex quadratic, hence symmetric, PSD, with constants in the kernel.

9.2 The general case

Each patch contributes $W^{(\ell)} = A_\ell (G^{(\ell)})^\top G^{(\ell)} \in \mathbb{R}^{(k+1) \times (k+1)}$, scattered into the global indices. Same FEM assembly as classical case — just with patches as elements.

9.3 The action formula

$$(Wf)_0 = \sum_{\ell: 0 \in \mathcal{N}^+(\ell)} A_\ell (G^{(\ell)}_{:,0})^\top (G^{(\ell)} f|_{\mathcal{N}^+(\ell)}).$$

Read this as: for each patch whose stencil contains v_0 , compute the gradient at v_ℓ , multiply by the column of $G^{(\ell)}$ at v_0 (which encodes how $\hat{g}^{(\ell)}$ depends on f_0 via the chain rule), and area-weight. Sum across patches. Energetically: $(Wf)_0$ is the variation of the Dirichlet energy w.r.t. f_0 .

10 Patch-local training of an assembled- W loss

Train end-to-end on assembled W while keeping every input strictly patch-shaped: process not one patch but a *cluster* of $k+1$ patches around target vertex v_0 (the patch at v_0 + the patches at each of v_0 's neighbors).

Why this works.

$$(Wf)_0 = \sum_{\ell: 0 \in \mathcal{N}^+(\ell)} A_\ell (G^{(\ell)}_{:,0})^\top \hat{g}^{(\ell)},$$

where the sum is over patches whose stencils contain v_0 — exactly $\{0\} \cup \mathcal{N}(0)$. Bounded set. Assembled action at one vertex requires only the 1-ring of patches around it.

Per training step.

1. Sample a Monge patch $h(u, v)$.
2. Pick center v_0 and its neighbors; for each, get its own k -NN.
3. Run network on all $k + 1$ patches (one batched forward).
4. Compute analytic ground truth $\Delta_g u(v_0)$.
5. Compute predicted $(Wu)_0$ via the assembly formula.
6. Loss + backward.

$\$ \sim 21 \$patch forward passes per step, batchable. Inference unchanged.$

11 A pure-norm shortcut: $w_{ij} = \|G_{:,j}^{(i)}\|^2$?

Tempting to define w_{ij} from gradient stencil norms, but it fails:

- **Sign loss.** Cotangent weights are signed; norms aren't.
- **Misses cross-term structure.** W_{ij} involves *inner products* $\langle G_{:,i}^{(\ell)}, G_{:,j}^{(\ell)} \rangle$, which depend on directional alignment.
- **Wrong dimensional scaling.** Norms scale as $1/h$; Laplacians as $1/h^2$. Squared norms are dimensionally correct.

Refined: zero out cross terms. Keep only single-patch diagonal contributions to W_{ij} :

$$W_{ij}^{(i), \text{diag-only}} = -A_i \|G_{:,j}^{(i)}\|^2.$$

Negative-of-norm-squared. With symmetrization: $w_{ij} = -\frac{1}{2}(A_i \|G_{:,j}^{(i)}\|^2 + A_j \|G_{:,i}^{(j)}\|^2)$. This is a coherent “decoupled-neighbor” approximation of $G^\top M_F G$, 1-hop sparse, and connects to graph-Laplacian-style learned operators.

12 What the stiffness weights actually are when you assemble $G^\top M_F G$

For diagonal:

$$W_{aa} = \sum_{\ell: a \in \mathcal{N}^+(\ell)} A_\ell \|G_{:,a}^{(\ell)}\|^2.$$

For off-diagonal:

$$W_{ab} = \sum_{\ell: a, b \in \mathcal{N}^+(\ell)} A_\ell \langle G_{:,a}^{(\ell)}, G_{:,b}^{(\ell)} \rangle.$$

Each patch contributes a $(k+1) \times (k+1)$ Gram matrix $W^{(\ell)} = A_\ell (G^{(\ell)})^\top G^{(\ell)}$, scatter-summed into the global W . Standard FEM assembly with patches as elements.

In code:

```
W_local = area_i * (G_i.T @ G_i)    # (k+1, k+1)
W[ix_(nbrs_i, nbrs_i)] += W_local
```

End-to-end differentiable; structural guarantees inherited from per-patch contributions.

13 Continuous-space training: opportunities from analytic data

Beyond pointwise discrete training, the analytic Monge patches enable several continuous-space approaches:

1. **Sobolev losses.** Match operator error in L^2 over the full patch via quadrature.
2. **Spectral / functional supervision.** Match the LBO spectrum directly.
3. **Variational / energy training.** Match the Dirichlet form $f^\top W f \approx E(f)$.
4. **Numerical-analysis convergence guarantees.** Treat the network as a learned discretization scheme to be analyzed via classical consistency / stability / convergence.
5. **PDE-residual / weak-form losses.** Penalize discrete eigenvalue or heat-equation residuals.
6. **Kernel learning.** Predict a continuous heat-kernel-like operator rather than discrete weights.
7. **Continuous gradient supervision.** Match interpolated discrete gradients to analytic gradient fields in L^2 .

The unifying theme: stop comparing discrete predictions to discrete samples; compare to continuous objects.

14 Avenue 2: spectral / functional supervision

The ultimate test is the spectrum — eigenvalues, eigenfunctions, heat kernels. Train directly on these.

Setup options. **Option A:** Closed analytic surfaces (sphere, torus, ellipsoid) with known spectra. **Option B:** Monge patches with Dirichlet/Neumann BCs; ground truth via high-order Galerkin. **Option C:** Match the Dirichlet form across test functions (boundary conditions enter only via test choice).

Loss.

$$\mathcal{L}_{\text{spec}} = \sum_k w_k |\lambda_k^{\text{disc}} - \lambda_k^{\text{cont}}|^2 + \alpha \sum_k w_k d_M(\phi_k^{\text{disc}}, \phi_k^{\text{cont}})^2.$$

Wrinkles.

- Sign / ordering: use sign-aware distance or subspace projector distance.
- Backprop through eigendecomposition: numerically unstable for close eigenvalues; use truncated iterative methods (Lanczos, LOBPCG).
- Train on small surfaces, generalize via patches.

Heat kernel matching. A more robust alternative:

$$\mathcal{L}_{\text{heat}} = \sum_i \|K_{t_i}^{\text{disc}} - K_{t_i}^{\text{cont}}\|_F^2.$$

No sign issues, smooth in t , broadband supervision.

Connection to your CVPR 2026 paper. Your work learns eigenstructures from data; spectral-supervised LBO learning produces the operator that yields the right eigenstructures. The two are complementary — a layered theoretical story.

15 Avenue 4: the network as a learned discretization scheme

View the network as defining a map

$$\mathcal{D}_\theta : \{\Sigma, V_h\} \mapsto W_\theta(\Sigma, V_h)$$

and ask: what classical numerical-analysis properties does it have?

Three properties.

- **Consistency:** $\|W_\theta f - \Delta_g f\| = O(h^p)$ for some order p .
- **Stability:** bounded condition number scaling, no spurious eigenvalues.
- **Convergence:** solutions of discrete problems approach continuous ones.

Empirical characterization. Sample each Monge patch at multiple resolutions; fit error vs. h in log-log to extract empirical consistency rate. Almost no learned-Laplacian paper does this.

Provable bounds. In restricted settings (bounded C^k surfaces, bounded networks, quasi-uniform sampling), classical-FEM-style consistency arguments combined with universal approximation can yield error bounds. Genuinely original mathematics.

Multi-resolution loss. Train with samples at multiple h levels and penalize deviation from the target convergence rate p . Train the asymptotic behavior, not just pointwise accuracy.

Connection to your research. Pairing the Afalo et al. optimal-approximation framework (continuous) with provable-convergence learned operators (discrete) gives a complete principled story.

16 Avenue 3: variational training via the Dirichlet energy

The LBO is fundamentally defined by its bilinear form, not its pointwise action:

$$\mathcal{E}(f, h) = \int_M \langle \nabla_g f, \nabla_g h \rangle_g dV_g, \quad E(f) = \frac{1}{2} \mathcal{E}(f, f).$$

Train by matching this form.

Why structurally cleaner.

- **Symmetry automatic** (form is symmetric).
- **Variational properties for free** (real spectrum, well-posed Galerkin, etc.).
- **Coordinate-invariant** (form is intrinsically defined).
- **Galerkin theory applies directly** (Céa’s lemma gives quasi-optimal energy approximation).

Loss.

$$\mathcal{L}_{\text{var}} = \frac{1}{m} \sum_{\ell} (\widehat{E}^{(\ell)} - E^{(\ell)})^2.$$

Where $\widehat{E}^{(\ell)} = \frac{1}{2} f^{(\ell)\top} W f^{(\ell)} = \frac{1}{2} \sum_i A_i \|\hat{g}_i^{(\ell)}\|^2$. The discrete sum is literally the Riemann sum approximating the continuous integral.

Test function families. Polynomials (Stone–Weierstrass), Gaussian bumps (multi-scale), random low-frequency functions.

Galerkin connection. Matching the bilinear form means doing learned Galerkin discretization. Solutions of discrete problems inherit Céa-quasi-optimal convergence. This bridges learning and numerical analysis directly.

Layered with your CVPR work. Variational training $\rightarrow$ provably-convergent discrete operator. Eigenstructure learning on top of it $\rightarrow$ optimal eigenfunction approximation. Combined: a complete pipeline from raw geometry to optimal eigenstructures with theoretical backing.

17 Computing the continuous Dirichlet energy on a Monge patch

17.1 Setup

Monge patch $\mathbf{X}(u, v) = (u, v, h(u, v))$ over $U \subset \mathbb{R}^2$. Test function $f(x, y, z)$ in ambient coordinates. Want:

$$E(f) = \frac{1}{2} \int_{\Sigma} \|\nabla_g f\|_g^2 dV_g.$$

17.2 Pull back to parameter domain

First fundamental form:

$$g = \begin{pmatrix} 1 + h_u^2 & h_u h_v \\ h_u h_v & 1 + h_v^2 \end{pmatrix}, \quad \det g = 1 + h_u^2 + h_v^2.$$

Inverse:

$$g^{-1} = \frac{1}{\det g} \begin{pmatrix} 1 + h_v^2 & -h_u h_v \\ -h_u h_v & 1 + h_u^2 \end{pmatrix}.$$

Volume form: $dV_g = \sqrt{\det g} du dv$.

17.3 Restricted function and gradient

$\tilde{f}(u, v) = f(u, v, h(u, v))$. By the chain rule:

$$\tilde{f}_u = f_x + f_z \cdot h_u, \quad \tilde{f}_v = f_y + f_z \cdot h_v.$$

Squared gradient norm:

$$\|\nabla_g \tilde{f}\|_g^2 = g^{uu} \tilde{f}_u^2 + 2g^{uv} \tilde{f}_u \tilde{f}_v + g^{vv} \tilde{f}_v^2.$$

17.4 Final integral

$$E(f) = \frac{1}{2} \int_U \frac{(1 + h_v^2) \tilde{f}_u^2 - 2h_u h_v \tilde{f}_u \tilde{f}_v + (1 + h_u^2) \tilde{f}_v^2}{\sqrt{1 + h_u^2 + h_v^2}} du dv.$$

17.5 Numerical evaluation

Gauss–Legendre quadrature on U : 30×30 points gives $\sim 10^{-12}$ precision for smooth integrands. The integrand is closed-form for any analytic h and f , so no symbolic computation at runtime — just evaluate at quadrature points and sum with weights.

17.6 Worked example

$h(u, v) = \frac{1}{2}(u^2 + v^2)$ (paraboloid), $U = [-0.5, 0.5]^2$, $f = x$:

$$\|\nabla_g \tilde{f}\|^2 = \frac{1 + v^2}{1 + u^2 + v^2}, \quad E(x) \approx 0.4856.$$

Sanity check: flat limit $h = 0$ gives $E = 0.5$. The paraboloid version is slightly less because metric stretching reduces gradient norm. ✓

17.7 Implementation

SymPy for one-off exact computation (slow). **NumPy** / **JAX** / **PyTorch** for fast batched quadrature integrating with autodiff. The integrand is a few lines of tensor operations once $h, h_u, h_v, f, f_x, f_y, f_z$ are evaluated at quadrature points.

18 Forward-pass design: predict G or W ?

18.1 Two main options

Option A: Predict W directly (1-hop sparse, simpler, no structural guarantees beyond what’s learned).

Option B: Predict G , assemble $W = G^\top M_F G$ (2-hop denser, but symmetric / PSD / kernel-of-constants automatic, gradient available as byproduct, variational interpretation exact).

18.2 Recommendation: Option B

For your research direction (variational training, spectral / Galerkin guarantees, eigenstructure learning), Option B aligns best. Specifically:

- Variational loss $\widehat{E}(f) = \frac{1}{2} \sum_i A_i \|\hat{g}_i\|^2$ is the discrete Riemann sum approximating the continuous integral. Direct correspondence.
- Galerkin theory applies cleanly with $W = G^\top M_F G$.
- Eigenproblems are well-conditioned (symmetric, PSD).
- You also get G for downstream vector-field applications.

18.3 Mass matrix

Compute M_F analytically from local geometry (PCA-based tangent areas, Voronoi, density estimation) rather than learning it. The mass matrix has clear geometric meaning, and the analytic computation is straightforward.

18.4 Architecture sketch

1. Geometric preprocessing: estimate normal $\mathbf{n}_0$ via PCA; estimate area A_0 .
2. Network forward: input patch geometry, output $G^{(0)} \in \mathbb{R}^{3 \times (k+1)}$ with column-sum-zero constraint.
3. Optional tangency correction: penalize or project the normal component.
4. Per-patch contribution: $W^{(0)} = A_0(G^{(0)})^\top G^{(0)}$.
5. Global assembly: $W_{\text{global}} = \sum_\ell \mathbf{S}_\ell^\top W^{(\ell)} \mathbf{S}_\ell$.

19 Computing $\widehat{E}^{(\ell)}$: a clarification

The W in $\widehat{E}^{(\ell)} = \frac{1}{2}(f^{(\ell)}|_V)^\top W(f^{(\ell)}|_V)$ is the *global assembled* stiffness on the entire surface, not the output of a single forward pass.

Per training step.

1. Sample one Monge surface Σ , n vertices V on it.
2. For each vertex v_i , run network forward $\rightarrow G^{(i)}$. Total: n forward passes (batched).
3. Compute analytic per-vertex areas $\{A_i\}$.
4. For each test function $f^{(\ell)}$:
 - Sample $f^{(\ell)}$ at all V .
 - $\hat{g}_i^{(\ell)} = G^{(i)} f^{(\ell)}|_{\mathcal{N}^+(i)}$ for $i = 1, \dots, n$.
 - $\widehat{E}^{(\ell)} = \frac{1}{2} \sum_i A_i \|\hat{g}_i^{(\ell)}\|^2$.
 - $E^{(\ell)} = \frac{1}{2} \int_\Sigma \|\nabla_g f^{(\ell)}\|^2 dV_g$ (quadrature).
5. Loss $\mathcal{L} = \sum_\ell (\widehat{E}^{(\ell)} - E^{(\ell)})^2$.
6. Backward through all n forward passes, update.

The energy is a global scalar aggregating across the whole surface — directly comparable to the continuous integral. It’s a Riemann-sum approximation, and the loss asks the network to make this approximation as accurate as possible across many test functions.

Cost variant. Sub-sample vertices per step for a Monte Carlo estimator if n is large.

20 Does energy minimization guarantee tangent gradients?

No. The energy loss only constrains $\sum_i A_i \|\hat{g}_i\|^2$ across test functions — it’s invariant under per-vertex per-function rotations of $\hat{g}_i$. The network can produce “gradients” that are rotated, partially-normal, or otherwise geometrically wrong as long as integrated squared norms work out.

To get tangent gradients, you need additional supervision.

- **Tangency penalty:** $\sum_i A_i \langle \hat{g}_i, \mathbf{n}_i \rangle^2$.
- **Architectural tangency:** predict in 2D tangent frame, lift to 3D.
- **Bilinear-form supervision:** match $f^\top W h \approx \int \langle \nabla f, \nabla h \rangle$ across pairs — pins down inner-product structure (still has per-vertex frame freedom).
- **Pointwise vector supervision:** match $\hat{g}_i$ to analytic $\nabla_g f(v_i)$ as vectors — pins down everything.

In practice, combine: variational + tangency penalty + (optional) pointwise vector loss. Energy alone is too weak; pointwise alone misses variational structure.

Generalization caveat. Even with all losses, inference correctness depends on training-distribution coverage. Sample diverse Monge patches, include edge cases (high curvature, anisotropic sampling, sharp features).

21 Bilinear-form vs. energy supervision

Yes: bilinear-form supervision is mechanically the energy procedure with two test functions instead of one.

Precisely.

- *Energy:* $\hat{E}(f) = \frac{1}{2} f^\top W f \approx E(f)$.
- *Bilinear:* $\hat{\mathcal{E}}(f, h) = f^\top W h \approx \mathcal{E}(f, h)$.

Computationally nearly identical: same forward passes, same quadrature, just compute pairwise inner products instead of diagonal squared norms.

Gauge analysis.

- Energy loss is invariant under per-function per-vertex rotations.
- Bilinear-form loss is invariant under per-vertex frame rotations only (much smaller gauge group).

So bilinear-form pins down gradient inner products and magnitudes; per-vertex frame still free. Doesn't directly enforce tangency, but pins down all entries of W (not just the symmetric quadratic part).

Polarization. If the test function set is closed under linear combinations, energy supervision implies bilinear supervision. For enumerated monomials (not closed), include sums or directly supervise pairs.

Hierarchy of training signals.

- Energy: pins per-vertex gradient magnitudes; per-function per-vertex rotation freedom.
- Bilinear form: pins per-vertex inner products; per-vertex frame freedom.

- Pointwise norm: pins per-vertex magnitudes exactly; per-vertex per-function rotation freedom.
- Pointwise vector: pins everything; no freedom.

Practical sweet spot: bilinear-form supervision + tangency penalty.

22 Choice of Monge surface family

Polynomials are fine for starting, but better options exist for production training:

Random Fourier features / GP samples. Primary backbone:

$$h(u, v) = \sum_{|k_1|+|k_2|\leq K} \frac{a_{k_1, k_2}}{(1 + |k_1|^2 + |k_2|^2)^{\beta/2}} [\cos(\pi k_1 u + \pi k_2 v) + \sin(\dots)].$$

Multi-scale, controllable spectrum, closed-form derivatives.

Sums of radial bumps.

$$h(u, v) = \sum_j A_j \exp\left(-\frac{(u - u_j)^2 + (v - v_j)^2}{\sigma_j^2}\right).$$

Localized high-curvature events; great complement to GPs.

Polynomial Monge patches. Smooth slow-varying patches; useful for low-degree ground truth.

Named analytic surfaces. Sphere caps, saddles, ellipsoids — for validation against known spectra.

Recommended training distribution. Mix random Fourier series, radial bumps, low-degree polynomials, and occasional named surfaces with curriculum from low-curvature to high-curvature.

Pitfalls. Normalize $\|h\|_\infty, \|\nabla h\|_\infty$ bounded; reject near-degenerate metrics; vary domain shape, sampling density, and noise; track curvature distribution to ensure coverage.

23 Implementing GP-like sampling

You don't need an external library. Random Fourier features (or direct trigonometric series with engineered spectrum) gives you:

- Closed-form samples.
- Closed-form (or autodiff) derivatives.
- Multi-scale by spectrum choice.
- Native GPU batching.
- 20 lines of NumPy/PyTorch code.

Sample implementation:


```

def generate_monge_patch(K=10, beta=2.0, device='cuda'):
    k1 = torch.arange(-K, K+1, device=device)
    k2 = torch.arange(-K, K+1, device=device)
    K1, K2 = torch.meshgrid(k1, k2, indexing='ij')
    K1, K2 = K1.flatten().float(), K2.flatten().float()
    spectral = 1.0 / (1.0 + K1**2 + K2**2)**(beta/2)
    a = torch.randn_like(spectral) * spectral
    b = torch.randn_like(spectral) * spectral
    def h(u, v):
        phase = torch.pi * (K1 * u.unsqueeze(-1) + K2 * v.unsqueeze(-1))
        return (a * torch.cos(phase) + b * torch.sin(phase)).sum(-1)
    return h

```

External libraries (GPyTorch, scikit-learn) are heavyweight and designed for problems you don't have (Bayesian inference, kernel learning, exact posterior sampling). Hand-rolled random Fourier features are the right tool.

Cost. Per training step, surface generation + ground truth computation under 1 ms; the network forward pass dominates.

Symbolic vs. autograd derivatives. For random Fourier series, derivatives are explicit (modified Fourier series); for general h , autograd is fine. Both compose naturally with the rest of the autodiff training pipeline.

End of conversation.